

```

timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
`include "alu.v"
`include "branch_decision.v"

```

```

module EX(
    input wire clk,
    input wire rst_n,
    input wire [4:0] a1,
    input wire [4:0] a2,
    input wire [31:0] i_rs1_data,
    input wire [31:0] i_rs2_data,
    input wire [31:0] i_Pc,
    input wire [3:0] i_Aluctrl,
    input wire [2:0] i_Func3,
    input wire [4:0] i_Rd,
    input wire i_RegSrc,
    input wire [31:0] i_Imm,
    input wire i_Sel1,
    input wire i_Sel2,
    input wire i_Load,
    input wire i_Branch,
    input wire i_Jal,
    input wire i_Jalr,
    input wire i_MemSrc,
    input wire [1:0] i_ResultSrc,
    input wire [4:0] i_amo_func5, // AMO instruction decode from ID stage
    input wire i_is_amo, // Flag if instruction is AMO

    output wire [31:0] o_DataStoreM,
    output wire [31:0] o_ResultM,
    output wire [2:0] o_Func3M,
    output wire [4:0] o_RdM,
    output wire o_RegSrcM,
    output wire o_MemSrcM,
    output wire [1:0] o_ResultSrcM,
    output wire [31:0] o_PcM,
    output wire o_LoadM,

    output wire [31:0] o_Result,
    output wire o_Boj,
    output wire o_Jalr,
    output wire [31:0] o_Imm_data,

    output wire [31:0] fwdwire1, fwdwire2,

```

```

    // Forwarding Unit
    input wire[1:0] i_mux1_sel,
    input wire[1:0] i_mux2_sel,
    input wire[31:0] i_ex_result,
    input wire[31:0] i_w_data
);

wire [31:0] alu_op1, alu_op2;
wire branch_decision;
reg [31:0] DataStore_reg, Result_reg;
reg [2:0] Func3_reg;
reg [4:0] Rd_reg;
reg RegSrc_reg, MemSrc_reg;
reg [1:0] ResultSrc_reg;
reg [31:0] Pc_reg;
reg Load_reg;

// Forwarding logic
assign fwdwire1 = (i_mux1_sel == 2'b01) ? i_ex_result : (i_mux1_sel == 2'b10) ?
i_w_data : i_rs1_data;
assign fwdwire2 = (i_mux2_sel == 2'b01) ? i_ex_result : (i_mux2_sel == 2'b10) ?
i_w_data : i_rs2_data;

// IF stage signal
assign o_Boj = (branch_decision & i_Branch) | i_Jal;
assign o_Jalr = i_Jalr;
assign o_Imm_data = i_Imm;

// MUX before ALU
assign alu_op1 = i_Sel1 ? fwdwire1 : i_Pc;
assign alu_op2 = i_Sel2 ? i_Imm : fwdwire2;

// Branch Decision Unit
branch_decision branch_dec_inst (
    .i_read_data1(fwdwire1),
    .i_read_data2(fwdwire2),
    .i_Func3(i_Func3),
    .o_Branch(branch_decision)
);

// AMO logic
reg [31:0] amo_result;
always @(*) begin
    case (i_amo_funct5)
        `AMO_ADD:    amo_result = fwdwire1 + fwdwire2;
    endcase
end

```

```

        `AMO_SWAP:    amo_result = fwdwire2;
        `AMO_XOR:     amo_result = fwdwire1 ^ fwdwire2;
        `AMO_AND:     amo_result = fwdwire1 & fwdwire2;
        `AMO_OR:      amo_result = fwdwire1 | fwdwire2;
        `AMO_MIN:     amo_result = ($signed(fwdwire1) < $signed(fwdwire2)) ? fwdwire1
: fwdwire2;
        `AMO_MAX:     amo_result = ($signed(fwdwire1) > $signed(fwdwire2)) ? fwdwire1
: fwdwire2;
        `AMO_MINU:    amo_result = (fwdwire1 < fwdwire2) ? fwdwire1 : fwdwire2;
        `AMO_MAXU:    amo_result = (fwdwire1 > fwdwire2) ? fwdwire1 : fwdwire2;
        `AMO_LR:      amo_result = fwdwire1; // Placeholder for Load-Reserved logic
        `AMO_SC:      amo_result = fwdwire2; // Placeholder for Store-Conditional logic
        default:      amo_result = 32'b0;
    endcase
end

// ALU instance
wire [31:0] alu_result;
alu alu_inst (
    .i_op1(alu_op1),
    .i_op2(alu_op2),
    .i_AluParam(i_AluParam),
    .o_Result(alu_result)
);

// Select final result (AMO or normal ALU)
assign o_Result = i_is_amo ? amo_result : alu_result;

// Pipeline Register Logic
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        DataStore_reg <= 32'b0;
        Result_reg <= 32'b0;
        Func3_reg <= 3'b0;
        Rd_reg <= 5'b0;
        RegSrc_reg <= 1'b0;
        MemSrc_reg <= 1'b0;
        ResultSrc_reg <= 2'b0;
        Pc_reg <= 32'b0;
        Load_reg <= 1'b0;
    end else begin
        Result_reg <= o_Result;
        DataStore_reg <= fwdwire2;
        Func3_reg <= i_Func3;
        Rd_reg <= i_Rd;
        RegSrc_reg <= i_RegSrc;
    end
end

```

```
        MemSrc_reg <= i_MemSrc;
        ResultSrc_reg <= i_ResultSrc;
        Pc_reg <= i_Pc;
        Load_reg <= i_Load;
    end
end

// Outputs to MEM stage
assign o_ResultM = Result_reg;
assign o_DataStoreM = DataStore_reg;
assign o_Func3M = Func3_reg;
assign o_RdM = Rd_reg;
assign o_RegSrcM = RegSrc_reg;
assign o_MemSrcM = MemSrc_reg;
assign o_ResultSrcM = ResultSrc_reg;
assign o_PcM = Pc_reg;
assign o_LoadM = Load_reg;

endmodule
```